

프로젝트

kt cloud 클라우드 인프라 부트캠프 2기에서 수행한 네 개의 팀 프로젝트 중, 제가 담당한 구현과 설계 판단을 중심으로 정리했습니다.

	포트폴리오 항목	핵심 내용	페이지
01	멀티클라우드 DR 운영·검증 자동화 팀 프로젝트: 멀티클라우드 이커머스 DR	인프라 수명주기·Failover/Failback 자동화와 접수 주문 정합성 검증	2
02	K3s·Argo CD GitOps 운영 구성 팀 프로젝트: 클라우드 네이티브 통합 보안 플랫폼	배포 의존성·인증 경로·워크로드 운영 설정과 보안 이벤트 연계	8
03	PR Preview 환경 생성·정리 자동화 팀 프로젝트: PR Preview 플랫폼	PR별 격리 환경 생성·배포와 Close/TTL 이중 정리	11
04	AWS 인프라 검증·장애·부하 테스트 팀 프로젝트: IaC 기반 클라우드 인프라 자동화	리소스 상태와 클라이언트 요청을 분리해 장애· 확장·성능 검증	13

기술 스택

영역	기술	적용	프로젝트
Kubernetes	K3s, EKS, AKS, Helm, HPA, KEDA	클러스터 구성, 워크로드 배치, 부하·이벤트 기반 확장	01 02 03
GitOps	Argo CD, Kustomize	Sync Wave 배포 순서 제어, Overlay 기반 DR 상태 관리	01 02
CI/CD · 자동화	GitHub Actions, GitHub App, Docker, Trivy	빌드·배포·인프라 생성·DR 전환, 자동화 권한 분리	01 02 03
클라우드	AWS (VPC, ALB, ASG, EKS, SSM, VPN), Azure (AKS, Key Vault), Cloudflare	네트워크·컴퓨트 구성, GSLB 라우팅 전환	01 02 03 04
IaC · 구성관리	Terraform, Ansible	검증용 환경 구성, 멀티클라우드 스택 실행 순서와 Output 연계	01 04
테스트 · 관측	pytest, Locust, k6, Prometheus, Grafana	리소스 상태 검증, 장애·부하 테스트, 메트릭 수집과 알림	01 02 04
보안 · 인증	External Secrets, IAM Role, Falco, ModSecurity	Secret 동기화, Access Key 없는 인증, 런타임·요청 이벤트 수집	01 02
데이터 · 메시징	PostgreSQL, Kafka	Replication과 Writer 전환, 이벤트 발행·소비, Replay	01 02

01 멀티클라우드 DR 운영·검증 자동화

팀 프로젝트 · 멀티클라우드 이커머스 DR

2026.04.16 - 07.21 · 7명

<https://github.com/z-kim/ktcloud-multi-cloud-dr>

가상의 이커머스 고객 환경을 대상으로, AWS 장애 상황에서도 접수 완료 주문을 보존하는 것을 우선 기준으로 두고 AWS Active-Azure Warm Standby DR을 설계했습니다. PostgreSQL logical replication으로 DB 상태를 Azure에 복제하고, 장애 시에는 Kafka Replay로 미처리 주문 이벤트를 다시 처리하도록 했습니다. DR 복구 완료 기준은 주문 수신 재개와 접수 완료 주문의 유실·중복 0건으로 정했습니다. 단일 DB Writer를 유지하도록 주문 수신·Consumer·Writer의 전환 순서를 정하고, 인프라 생성·삭제, Failover/Failback, 장애·복구 검증을 각각의 Workflow로 구현했습니다.

완료 기준

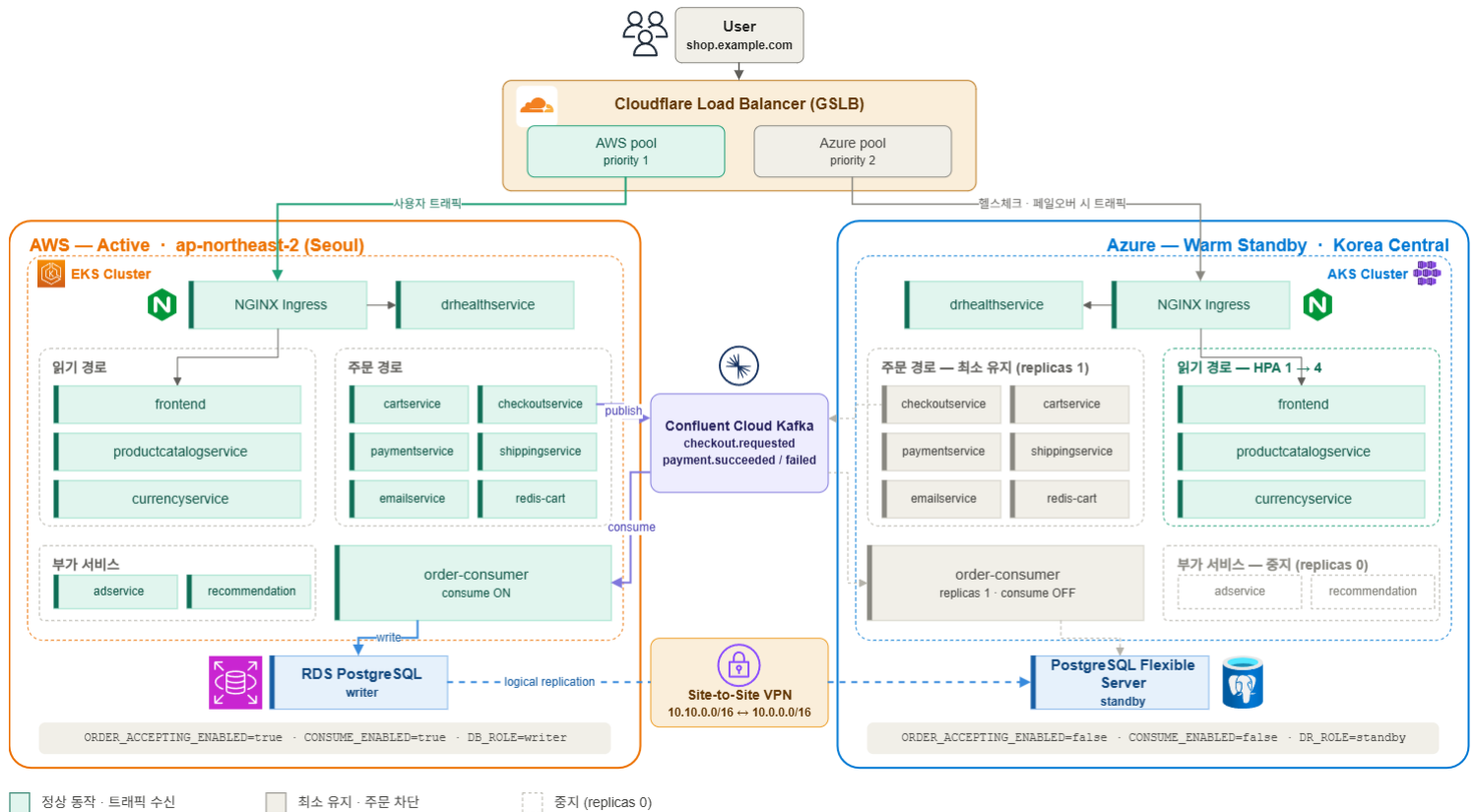
- AWS 장애 시 RTO 15분 이내
- 접수 완료 주문 기준 RPO 0(유실 0건) · 중복 0건

결과 요약 Failover 주문 불가 497초 · 접수 주문 유실 0건 · 중복 0건

담당 범위

7명 팀은 AWS 인프라 / Azure 인프라 / Cloudflare GSLB·장애 감지·트래픽 전환 / Kafka / PostgreSQL·Replication / GitOps·DR Workflow / PM으로 역할을 나눴습니다. 저는 GitOps·DR Workflow를 담당했습니다. 각 영역에서 필요한 입력값과 실행 순서를 정리하고, Deploy/Destroy, Failover/Failback, DR 검증 절차를 각각의 Workflow로 구현했습니다. 담당 범위는 다음과 같습니다.

- 인프라 생성·삭제 워크플로우 (Deploy All / Destroy All)
- GitOps 운영 상태 설계와 Overlay 구현
- DR Failover / Failback 자동화
- 정상·장애·DR 검증 자동화와 Drill 리포트
- 애플리케이션 부분 장애 격리



멀티클라우드 DR 전체 아키텍처

01-1 Warm Standby 설계

RTO 15분 요구사항과 상시 운영 비용을 고려해 Active-Active 대신 Azure Warm Standby를 선택했습니다. Azure에는 상품 조회부터 주문 완료까지 필요한 서비스만 상시 유지하고, 광고·추천 등 주문과 무관한 부가 서비스는 replica 0으로 설정했습니다.

checkout·payment·shipping·email·cart는 최소 replica로 유지했습니다. 상품 조회 요청을 처리하는 frontend·productcatalogservice·currencyservice는 평상시 replica 1로 유지하고, 전환 시에는 HPA로 최대 4개까지 확장되도록 구성했습니다.

01-2 인프라 수명주기 자동화

상시 운영 비용을 줄이기 위해 환경을 매일 생성·삭제했고, 여러 Terraform 스택의 실행 순서와 스택 간 전달값을 반복해서 관리해야 했습니다. 생성·삭제 절차를 각각 하나의 Workflow로 묶어, 같은 순서로 반복 실행할 수 있도록 했습니다.

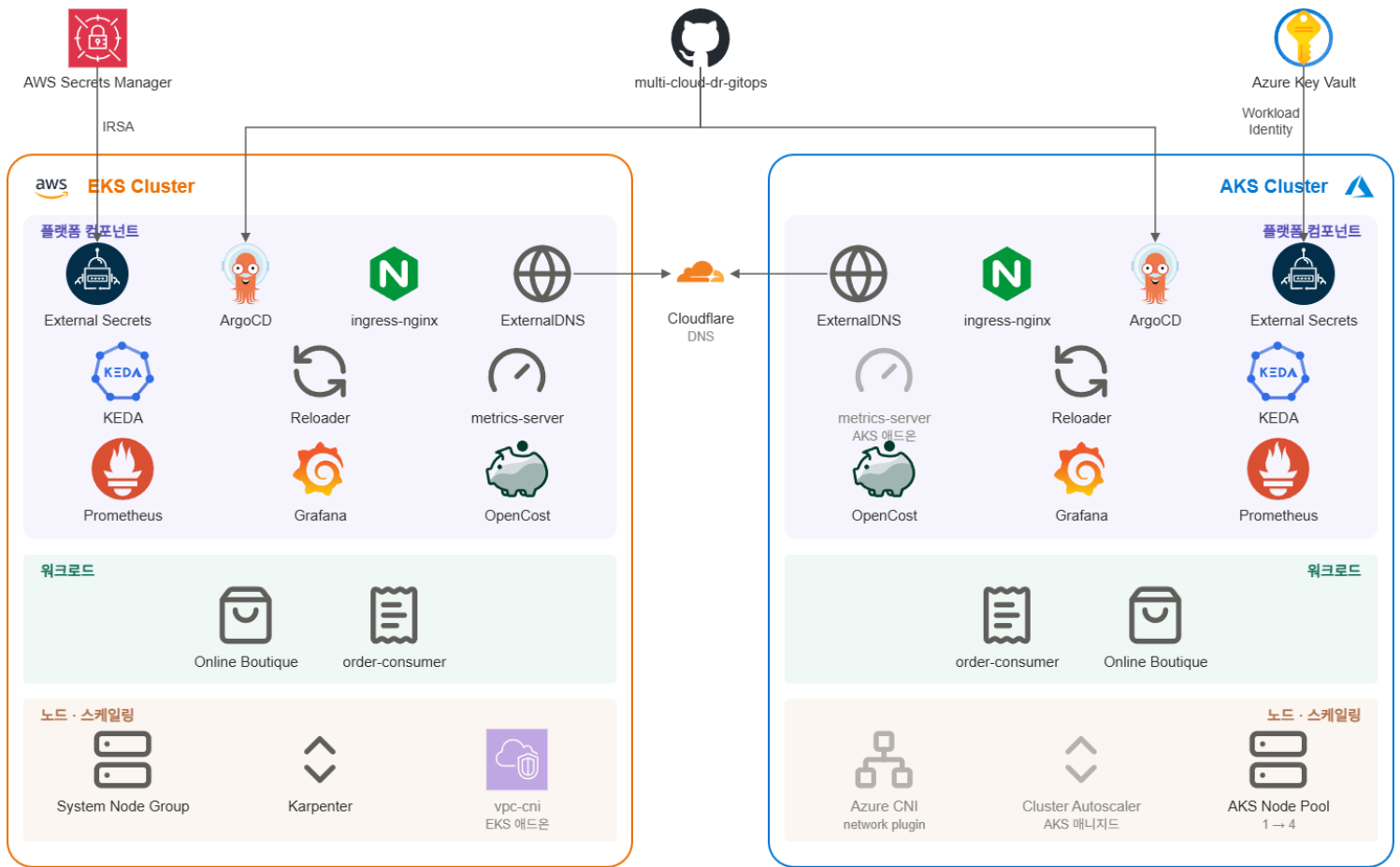
Deploy All은 Terraform 스택 실행, Runtime Secret 저장, GitOps 설정 반영, Argo CD bootstrap, DB 운영 준비를 순서대로 수행하도록 구성했습니다. **Destroy All**은 리소스 간 의존성과 외부 연결을 고려한 순서로 환경을 정리하도록 구성했습니다.

- AWS VPC/EKS/RDS, Azure VNet/AKS/PostgreSQL, Confluent Kafka, Cloudflare 스택 실행 순서 정의
- AWS-Azure site-to-site VPN과 PostgreSQL replication 절차를 구축·정리 워크플로에 연결
- Terraform output을 실행 환경 설정과 GitOps 운영 값으로 전달 (DB host, Kafka bootstrap, ExternalSecret 참조)
- AWS Secrets Manager · Azure Key Vault와 ExternalSecret 연계

삭제 절차 보완. Destroy All을 반복 실행하는 과정에서 AWS 인프라 삭제가 VPC 단계에서 중단됐습니다. 확인 결과, EKS가 생성한 network interface와 security group이 클러스터 삭제 후에도 남아 VPC 삭제를 막고 있었습니다. 남은 리소스를 정리하는 단계를 추가하되, 동작 중인 control plane의 리소스를 잘못 삭제하지 않도록 클러스터가 남아 있으면 정리를 건너뛰도록 했습니다. AWS 삭제 요청이 실제 리소스 상태에 반영되기까지 시차가 있어, 삭제 완료를 확인한 뒤 다음 단계로 진행하도록 했습니다.

01-3 EKS·AKS 런타임 구성

Argo CD, External Secrets 등 공통 플랫폼 구성요소는 EKS와 AKS 양쪽에 배치하고, 인증·Secret 저장소·노드 확장처럼 클라우드별로 다른 부분은 각 환경에 맞게 분리했습니다. GitOps 저장소는 하나로 두고 EKS·AKS 경로를 나눠 관리했습니다.



흐리게 표시된 항목은 클라우드가 매니지드로 제공합니다.

EKS · AKS 런타임 구성요소

- Argo CD와 External Secrets를 양쪽에 배치하고, Secret은 클라우드별 저장소(Secrets Manager / Key Vault)에서 IRSA·Workload Identity로 참조
- ingress-nginx와 ExternalDNS로 진입 경로를 구성하고 Cloudflare DNS에 등록
- HPA는 Metrics Server에서 수집한 CPU 지표, KEDA는 Kafka lag를 기준으로 동작하도록 구성
- Secret·ConfigMap 변경 시 Reloader가 워크로드를 갱신
- 노드 확장은 EKS에서 Karpenter, AKS에서 Cluster Autoscaler로 처리
- Prometheus·Grafana·OpenCost로 상태와 비용을 관측

01-4 DR 운영 상태와 전환 안전성

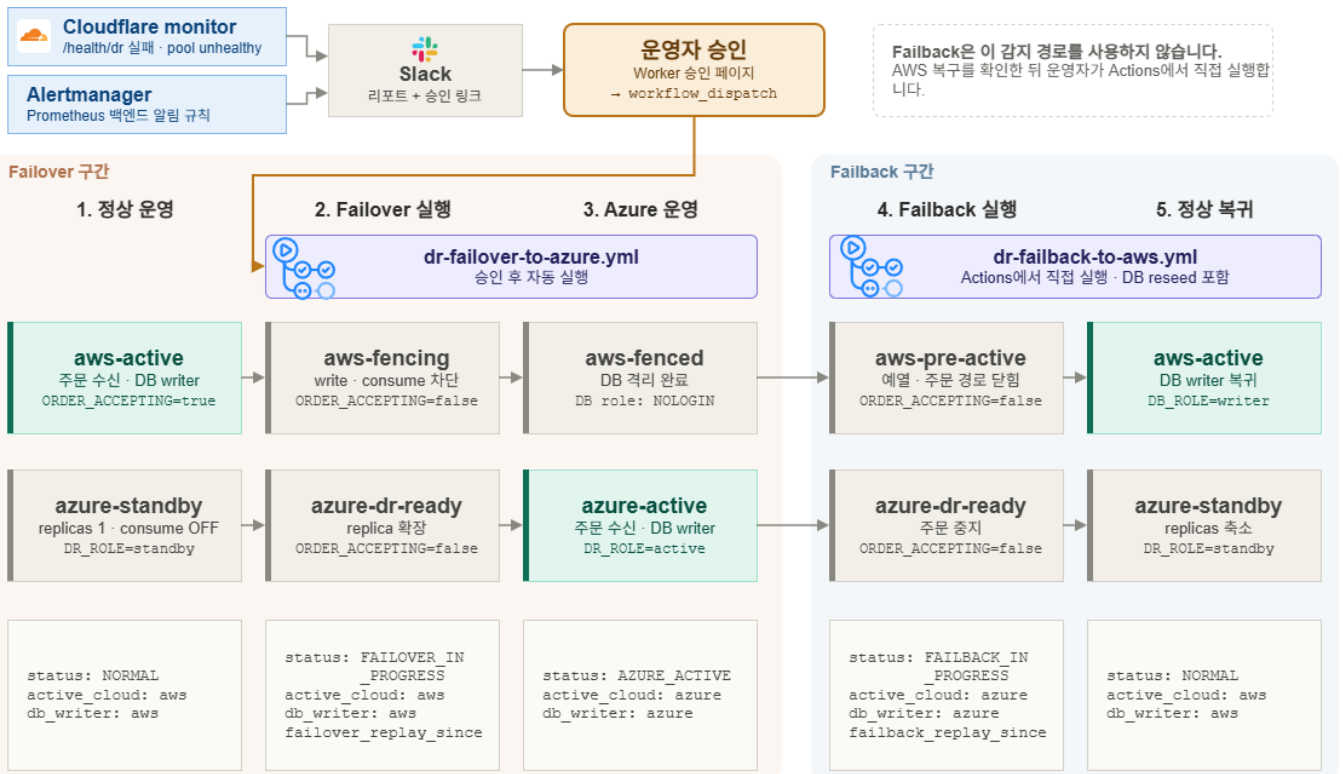
DR 전환 중간 상태는 Active/Standby 두 값만으로 표현하기 어려웠습니다. Azure의 서비스 준비 완료와 주문 처리 활성화 시점을 분리하고, 주문 처리는 DB Writer 전환과 Replay가 끝난 뒤에만 활성화했습니다. 전환 상태는 주문 수신·Consumer·Writer·Scaling을 각각 제어할 수 있도록 7개 Overlay로 나눴습니다.

	상태	내용
AWS	aws-active	주문 수신 · DB writer
	aws-fencing	write · consume 차단 진행
	aws-fenced	DB role NOLOGIN · 격리 완료
	aws-pre-active	복귀 준비 · 주문 경로는 닫힘
Azure	azure-standby	최소 replica · consume OFF
	azure-dr-ready	replica 확장 · 주문 수신 전
	azure-active	주문 수신 · DB writer

DR Workflow에서는 Argo CD Application path를 바꿔 각 클러스터의 상태를 전환했습니다.

active-core 공통 계층. aws-pre-active와 azure-dr-ready는 각 클라우드의 active 상태와 같은 replica·HPA 구성을 사용하되 주문 처리는 비활성화했습니다. 각 클라우드에서 공통으로 사용하는 replica·HPA 설정은 active-core로 묶고, 주문 수신 flag와 Kafka lag 기반 KEDA 등 상태별 차이만 최종 Overlay에서 제어했습니다.

장애 감지 · 승인



DR 상태 전이 다이어그램

자동 전환 범위와 승인 경계

Cloudflare Monitor로 장애 감지와 트래픽 라우팅을 자동화하고, Alertmanager·Slack으로 장애를 알렸습니다. DB Writer 전환·AWS fencing·Kafka Replay·active 승격처럼 데이터 상태를 바꾸는 단계는 운영자 승인 후 실행했습니다.

Slack으로 전달되는 장애 보고서에 Failover 승인 URL을 포함하고, 장애 정보와 Replay 기준 시각을 함께 전달해 승인 후 필요한 값을 다시 입력하지 않도록 했습니다.

Write path 이중 차단

애플리케이션 flag 외의 DB 접근 경로도 차단하기 위해 주문 수신 flag와 DB writer role의 LOGIN / NOLOGIN을 함께 사용했습니다. 애플리케이션 계층과 DB 계층에서 write path를 이중으로 제어해 양쪽 클라우드의 동시 write를 방지했습니다.

DB 접근 권한은 실행 목적에 따라 분리했습니다. 애플리케이션 Runtime은 order_consumer_writer를 사용했습니다. migration·backup·exporter 작업은 db-ops, writer 전환과 replication 설정·정리는 db-admin-ops로 분리했습니다.

전환 상태 기록

Failover/Failback이 여러 Workflow로 나뉘어 있어 다음 실행에서 현재 DR 상태를 판단할 기준이 필요했습니다. dr-state/current.json에 현재 전환 상태, Active Cloud, DB Writer, 진행 중인 incident와 Replay 기준 시각을 기록했습니다. 각 Workflow는 시작할 때 이 상태를 확인해 중복 실행과 잘못된 방향의 전환을 차단했습니다.

01-5 운영 Workflow와 권한 분리

실제 Failover/Failback과 장애 주입·부하 생성은 목적과 위험이 달랐습니다. 운영 Workflow에는 GitOps 상태 전환, DB Writer 제어, Kafka Replay, Cloudflare 복구만 두고, 장애 주입·Traffic 관측·Cleanup·Summary는 별도의 DR Drill로 분리했습니다.

GitHub App 기반 권한 분리

GitOps 저장소 변경과 DR 상태 기록, 애플리케이션 저장소 조회, Cloudflare Worker의 DR Workflow 실행 요청을 각각 별도의 GitHub App으로 분리했습니다. 각 App에는 해당 작업에 필요한 최소 권한만 부여했습니다.

01-6 애플리케이션 부분 장애 격리

장애 검증 과정에서 클라우드 전환이 필요하지 않은 애플리케이션 부분 장애도 확인했습니다. currency-service 장애가 상품 목록·상세·장바구니 전체의 500 오류로 전파되어, 표시용 환율 변환 실패만으로 주문과 무관한 화면까지 중단됐습니다.

- 환율 변환 실패 시 기준 통화로 표시를 유지하고 화면은 정상 렌더링
- 상품 목록·상세에서는 장바구니 조회가 실패해도 개수만 비우고 화면을 유지, 장바구니 화면에서는 오류로 처리
- 주문 실패를 원인별로 구분(접수 비활성화·주문 준비 실패·이벤트 발행 실패·결제 실패)해 각각 다른 안내와 상태 코드로 응답
- 오류 화면에는 내부 오류 문자열 대신 안내 문구와 request id를 노출하고, 상세 정보 노출 여부는 환경 변수로 제어

화면별로 필요한 데이터 수준에 따라 fallback 범위를 달리했습니다. 상품 목록·상세에서는 장바구니 개수 조회 실패를 허용하고, 장바구니 화면에서는 같은 실패를 오류로 처리했습니다.

01-7 DR 검증과 실행 결과

DR 전환 후에는 Kubernetes 상태, 주문 처리, 데이터 보존을 각각 확인해야 했습니다. 이 항목들을 반복해서 확인할 수 있도록 Runtime, 주문 E2E, DB 정합성, Autoscaling, 장애 알림, Failover/Failback 시나리오를 자동화했습니다.

검증 범위

런타임·준비 상태	Deployment·Pod Ready, Replica 수, Argo CD Sync/Health, ExternalSecret 동기화, Ingress 외부 접근
주문 경로·데이터	checkoutservice → Kafka → order-consumer → PostgreSQL 전 구간 확인, 주문 중복 저장 대조
확장·장애 대응	HPA·KEDA 스케일 동작, checkoutservice·Kafka·DB 장애 알림, 라우팅 전환·복구
DR 실행·리포트	Failover·Failback 실행, RTO·RPO 및 Accepted 주문 유실·중복 검증

DR Drill의 장애 주입, 알림, Traffic Driver, Failover/Failback, Cleanup, Summary는 각각 별도의 Workflow로 분리했습니다. 모든 Workflow에 같은 incident_id를 전달해 summary와 artifact를 Drill 단위로 묶었습니다.

Accepted 주문 기준 보존 검증

개별 접수 주문의 보존 여부를 확인하기 위해 Traffic Driver가 Accepted로 기록한 주문 ID를 별도로 저장하고, Failover/Failback 이후 최종 DB와 대조했습니다. 접수·거절·실패를 구분해 기록하고, 접수 완료 주문 기준으로 유실과 중복을 검증했습니다.

DR Drill 실행 결과

항목	결과
Drill 결과	PASS
부하 조건	failover/failback 각 20분, VUs 4, order probability 0.45
주문 불가 구간	failover 497초 / failback 703초
주문 시도 / 접수 / 거절 / 실패	3,329 / 1,392 / 1,520 / 417
접수 주문 유실 / 중복	0 / 0
RPO	0 (접수 완료 주문 기준)

Failover에서 장애 발생 후 주문 수신이 재개되기까지 497초가 걸려 목표 RTO 15분을 충족했습니다. 그 밖에도 다음 항목을 검증했습니다.

- Normal order E2E — 시도 61건, 접수 61건, 실패 0
- HPA — frontend 2 → 4
- KEDA — order-consumer 2 → 4, checkout.requested 1,200건 burst
- Backend Alert Drill — checkoutservice 장애 주입 후 Alertmanager 알림·복구 확인

01-8 한계와 개선 방향

Failover 대상 장애 구분. 프로젝트에서는 Cloudflare Pool Health와 EKS 상태를 중심으로 장애를 감지했습니다. Kafka처럼 두 클라우드가 공유하는 구성요소의 장애는 클라우드 전환만으로 해결되지 않습니다. Failover 전에 외부 요청과 주요 의존성 상태를 확인하는 사전 점검(Precheck)을 추가하면, 클라우드 전환이 필요한 장애인지 먼저 구분할 수 있습니다.

주문 수신 재개 시점. DR Drill에서 Cloudflare 라우팅은 장애 감지 후 25초 만에 Azure로 전환됐지만 주문 수신 재개까지 497초가 걸렸습니다. DB Writer 전환과 Kafka Replay·정합성 확인이 끝난 뒤 주문을 다시 열었기 때문입니다. 장애 시점의 Kafka offset을 복구 기준으로 고정하고, Replay·정합성 검증이 끝나면 신규 주문 수신을 먼저 재개한 뒤 남은 backlog를 Consumer가 처리하도록 분리하는 방안을 검토했습니다.

여러 Workflow의 실행 상태 통합. DR Drill이 여러 GitHub Actions Run으로 나뉘어 진행 상태와 결과가 분산됐습니다. 실행 순서·승인·상태·artifact·summary를 별도 Orchestrator에서 관리해 전체 Drill을 한곳에서 확인하는 구조를 검토했습니다.

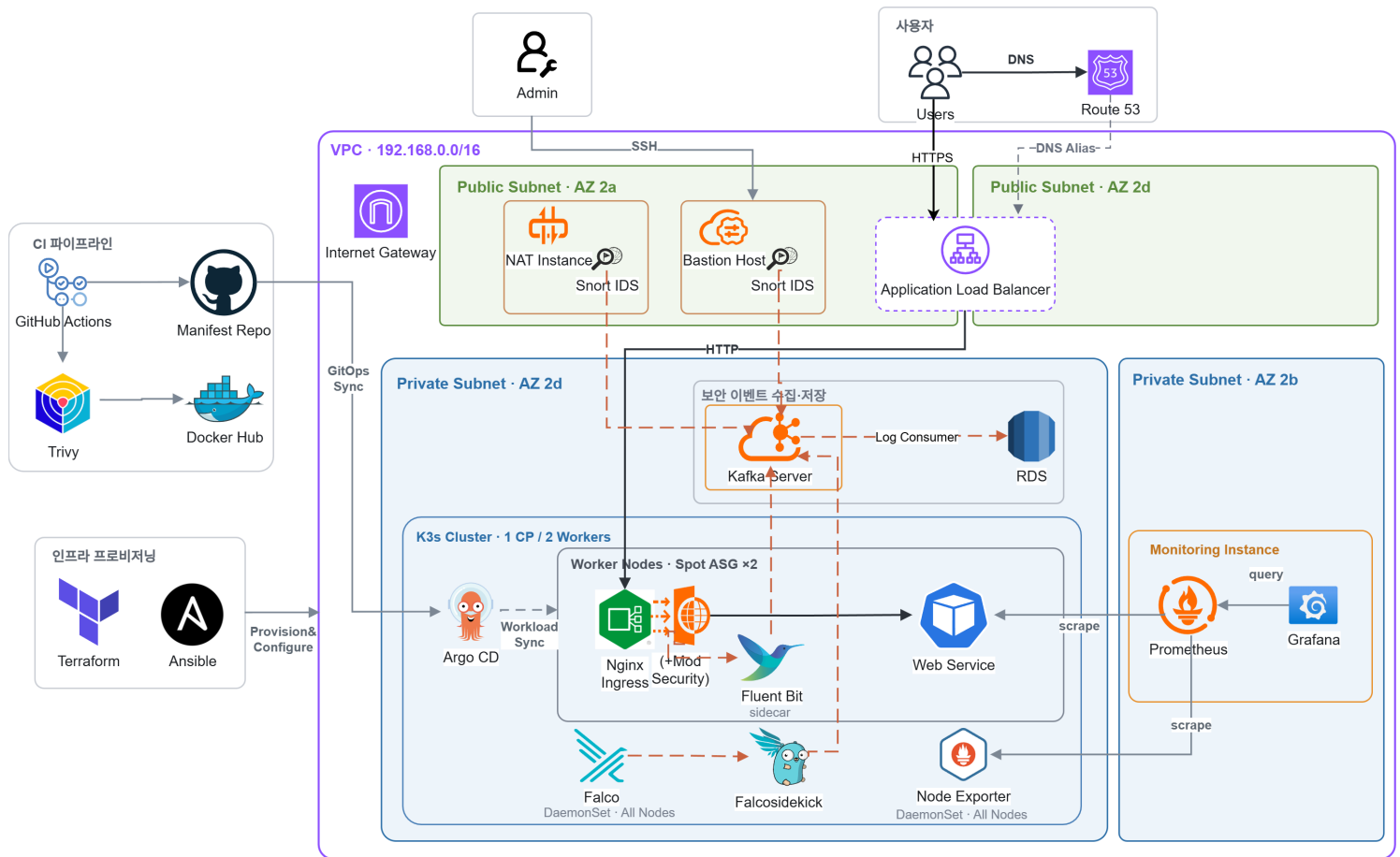
02 K3s · Argo CD GitOps 운영 구성

팀 프로젝트 · 클라우드 네이티브 통합 보안 플랫폼

2026.03.10 - 04.09 · 6명

<https://github.com/z-kim/ktcloud-sixsense-k3s-cluster>

K3s 기반 클라우드 네이티브 통합 보안 플랫폼 프로젝트에서 K3s 실행 환경, Kubernetes 워크로드 설정, Argo CD 기반 GitOps 배포를 담당했습니다. 프로젝트 전체에서는 Snort · Falco · ModSecurity로 네트워크 · 런타임 · 요청 계층의 보안 이벤트를 탐지하고, Trivy로 컨테이너 이미지 취약점을 검사했습니다. 저는 ModSecurity · Falco 이벤트를 Kafka로 전달하는 수집 경로도 구성했습니다.



K3s · GitOps · 보안 이벤트 수집 아키텍처

02-1 K3s 실행 환경 보완

초기 K3s 구성에서 Worker 등록과 노드 간 통신 문제가 있어 관련 설정을 보완했습니다. Worker ASG, k3s_token, control-plane Ansible 설정과 노드 통신 Security Group을 조정해 1 Control Plane / 2 Worker 구성을 확인했습니다.

02-2 GitOps 전환과 적용 순서 설계

초기에는 Manifest를 필요한 순서대로 직접 적용했고, 수동 적용을 반복하면서 적용 순서 관리와 설정 누락 문제가 발생했습니다. Argo CD App-of-Apps로 전환해 구성요소를 Application 단위로 나누고, 배포 의존성과 운영상 선후관계를 기준으로 Sync Wave를 지정했습니다.

Application 단위 — Sync Wave. 외부 진입 경로와 기본 관측 구성을 먼저 적용하고, 런타임 보안 구성 이후 애플리케이션을 적용하도록 순서를 나눴습니다.

Wave	Application	배치 기준
1	ingress-nginx, node-exporter	외부 진입 경로 · 기본 관측 구성 먼저 적용
2	falco	애플리케이션 배포 전 런타임 보안 구성 적용
3	doc-converter	공통 인프라 · 보안 구성 이후 앱 적용

리소스 단위 — HelmChartConfig와 HelmChart. HelmChart가 HelmChartConfig보다 먼저 적용되면 보안 · 로그 설정이 반영되지 않은 기본값으로 설치될 수 있습니다. HelmChartConfig에 sync-wave: -1, HelmChart에 0을 지정해 HelmChartConfig가 먼저 적용되도록 했습니다.

클러스터 단위 — Bootstrap. namespace, Repository Secret, kafka-alias처럼 child Application이 참조하는 리소스는 root-app보다 먼저 준비되어야 했습니다. 아래 순서로 Bootstrap 절차를 나누고 각 단계의 선행 조건을 확인했습니다.

namespace → External Secrets → Argo CD → bootstrap 입력 리소스 → root-app

- namespaced 리소스를 적용하려면 namespace가 먼저 생성되어야 합니다.
- Argo CD가 repository를 읽으려면 argocd-root-repo Secret이 준비되어야 합니다.
- kafka-alias는 ingress-nginx sidecar와 Falco가 참조하므로 child app Sync 이전에 필요합니다.
- root-app은 마지막에 적용해야 child app이 선행 리소스를 인식합니다.

재실행 시에도 선행 조건을 다시 확인했습니다. 입력 파일을 먼저 검증하고, External Secrets · Argo CD · repo Secret이 준비된 뒤 다음 단계로 넘어가도록 했습니다. 대기 시간이 timeout을 넘으면 관련 리소스 상태를 출력하도록 했습니다.

02-3 인증 경로와 Runtime Secret 분리

Private Repository 인증에는 개인 PAT 대신 읽기 전용 GitHub App을 사용했습니다. App ID · Installation ID · Private Key는 AWS SSM Parameter Store에 저장하고, External Secrets를 통해 Argo CD Repository Secret으로 주입했습니다. AWS 접근에는 장기 Access Key 대신 노드 IAM Role을 사용했습니다. control-plane Role은 Argo CD 자격정보 조회에, worker Role은 애플리케이션의 S3 접근에 사용했습니다.

02-4 보안 이벤트 수집 경로

ModSecurity 감사 로그와 Falco 런타임 이벤트를 같은 Kafka 경로로 모았습니다. 수집 설정은 실제 Kafka Server 주소 대신 클러스터 내부의 kafka.logging을 참조하도록 했습니다.

ModSecurity	Ingress 요청 → 검사 → audit log → fluent-bit sidecar → kafka.logging → Kafka Server
Falco	노드 런타임 이벤트 → Falco → Falcosidekick → kafka.logging → Kafka Server

실제 Kafka Server 주소는 Service와 EndpointSlice에서 관리해, 주소가 바뀌어도 ModSecurity와 Falco의 수집 설정은 수정하지 않도록 했습니다.

02-5 환경별 설정 분리와 배포 안정성

공통 Manifest가 중복되지 않도록 base와 overlay로 나눴습니다. Pod 분산·Probe·Rolling Update 정책도 Manifest에 포함해 배포 설정과 함께 관리했습니다.

- resource·toleration·image tag·metrics NodePort 등 환경별 값만 overlay에서 관리
- anti-affinity·topology spread로 Pod를 분산하고 startup·readiness·liveness probe를 적용, Rolling Update는 maxUnavailable: 0 / maxSurge: 1로 설정
- 이미지는 github.sha로 태깅해 Kustomize newTag를 갱신하고, CRITICAL 취약점 발견 시 Trivy 단계에서 워크플로 중단

02-6 실행 결과

실제 환경에서 K3s 1 Control Plane / 2 Worker 구성과 노드 간 통신을 확인했습니다. Argo CD 배포는 Sync Wave와 Bootstrap 선행 조건에 따라 순서대로 적용되는 것을 확인했고, Private Repository 인증과 ModSecurity·Falco 이벤트의 Kafka 수집 경로도 정상 동작을 확인했습니다. 또한 doc-converter에 Pod 분산과 Rolling Update 설정이 적용된 상태로 배포되는 것을 확인했습니다.

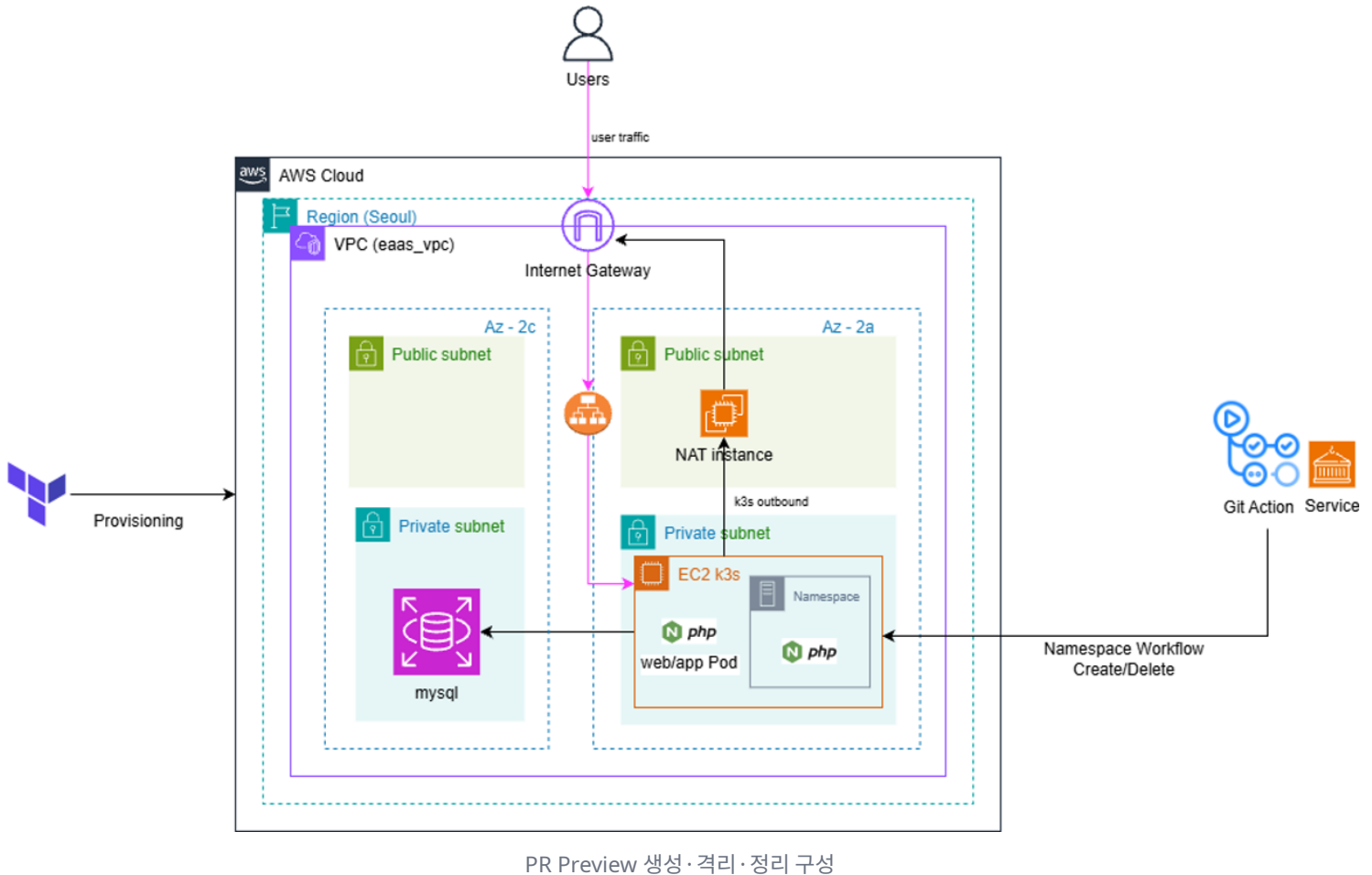
03 PR Preview 환경 생성·정리 자동화

팀 프로젝트 · PR Preview 플랫폼

2026.04.13 - 04.15 · 4명 · 3일 해커톤

<https://github.com/z-kim/ktcloud-hackaton-a-cicd>

리뷰어가 변경사항을 확인할 때 별도 배포 환경을 준비하지 않도록, PR마다 격리된 Preview 환경을 자동 생성했습니다. PR Open 시 애플리케이션과 DB를 배포하고, PR Close 시 환경을 정리하도록 구성했습니다. 3일간의 해커톤에서 GitHub Actions 기반 Preview 배포와 Close/TTL 정리를 담당했습니다.



03-1 PR 번호 기반 환경 격리

Namespace, DB, Host, image tag에 모두 PR 번호를 사용했습니다.

Namespace	preview-pr-<번호>
Database	pr_<번호>
Host	pr-<번호>.<base-domain>
Image tag	pr-<번호>-<short sha>

동일 PR에 새 커밋이 추가될 때는 기존 환경을 갱신하고, PR 번호를 Namespace·DB·Host의 공통 식별자로 유지했습니다.

03-2 SSM 기반 비공개 클러스터 배포

GitHub-hosted runner에서 비공개 K3s에 접근해야 했습니다. API 서버를 외부에 노출하는 대신 AWS SSM SendCommand를 배포 채널로 사용했습니다. runner에는 AWS 자격증명만 제공하고, Kubernetes 조작은 K3s 서버 내부에서 실행했습니다.

03-3 정리 경로 이중화

PR Close 이벤트만으로는 Workflow 실패나 이벤트 누락 시 Preview 환경이 남을 수 있어 정리 경로를 두 단계로 나눴습니다. PR Close 시 DB cleanup Job을 실행한 뒤 namespace를 삭제하고, 누락된 정리는 TTL 기반 CronJob으로 처리했습니다.

CronJob은 5분마다 preview-environment=true namespace를 조회해, expires-at annotation이 지난 환경의 DB와 namespace를 정리하도록 했습니다. Close Workflow가 실행되지 않아도 만료 시점 이후 정리되도록 했습니다.

03-4 설정 참조 지점 단일화

도메인 값을 여러 Manifest에 직접 넣지 않고 PREVIEW_BASE_DOMAIN으로 관리했습니다. Ingress 템플릿·PR 댓글 URL·앱 Manifest·DB Secret이 이 값을 참조하도록 해, 도메인 변경 시 수정할 위치를 Workflow 2곳과 wildcard DNS 1곳으로 줄였습니다.

03-5 실행 결과

- PR 생성·업데이트 시 Preview 환경 자동 생성·갱신, PR 댓글에 URL과 만료 시각 기록
- 여러 PR의 Namespace·DB·Host 동시 격리 운영
- PR Close 시 전용 DB·Namespace 즉시 정리, 이벤트 누락 시 TTL 후속 정리
- K3s API 외부 노출 없이 배포·삭제 완료

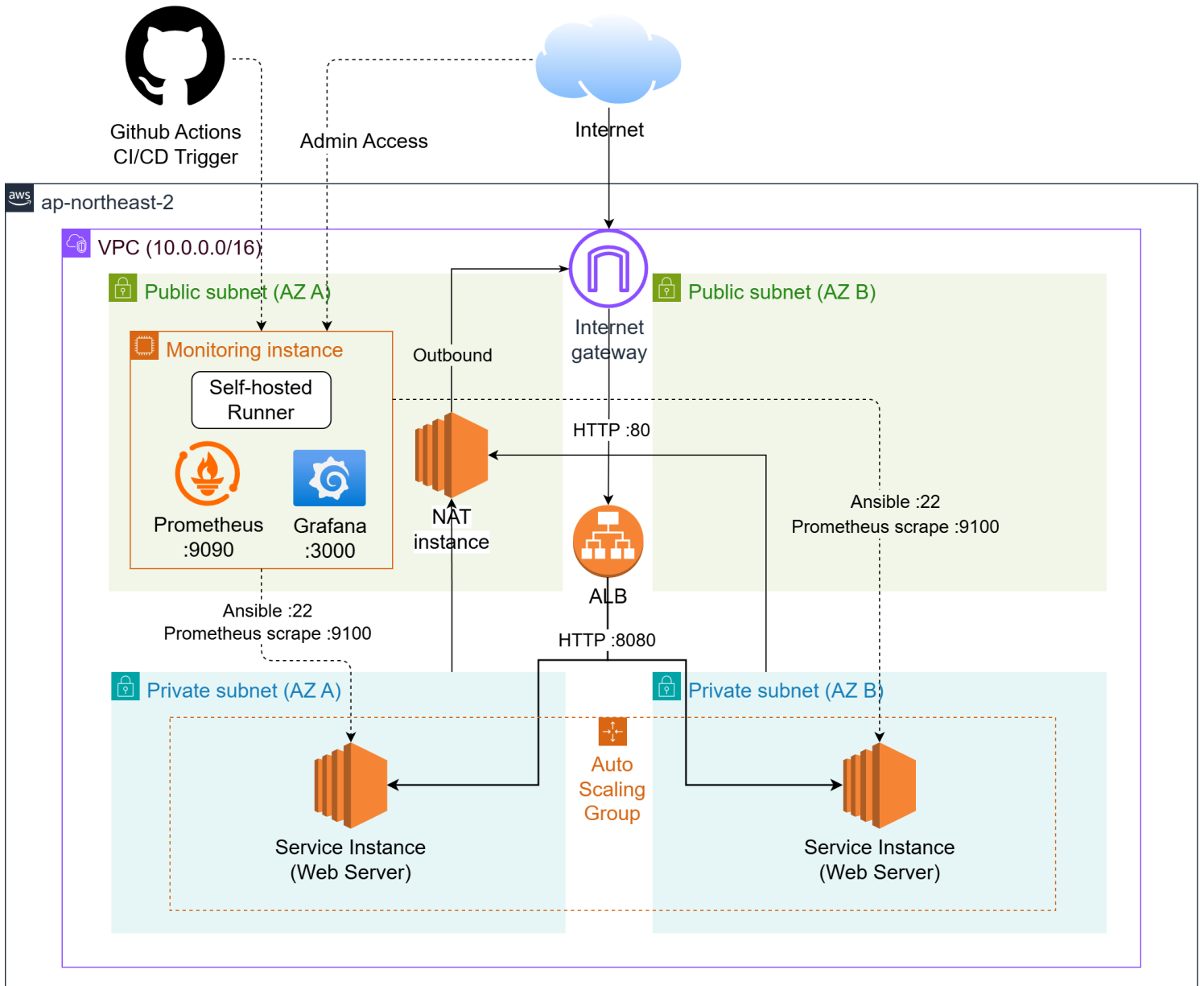
04 AWS 인프라 검증 · 장애 · 부하 테스트

팀 프로젝트 · IaC 기반 클라우드 인프라 자동화

2026.01.26 - 03.04 · 5명

<https://github.com/z-kim/ktcloudmini-aws-infra-tests>

Terraform · Ansible · GitHub Actions로 AWS 웹 인프라를 구성한 팀 프로젝트에서 인프라 검증과 장애 · 부하 테스트를 담당했습니다. 정상 · 장애 · 부하 조건을 같은 기준으로 반복 검증할 수 있도록 테스트를 코드화했습니다. pytest · boto3로 인프라 구성과 동작을 확인하고, Locust로 클라이언트 요청의 성공률과 응답 성능을 측정했습니다.



검증 대상이 된 팀 AWS 인프라 — ALB · ASG · EC2 · Prometheus

04-1 검증 로직과 대상 인프라 분리

ALB · ASG · Target Group 정보는 테스트 코드에 고정하지 않고 Terraform Output 또는 환경 변수에서 읽도록 구성했습니다. ALB_URL, ASG_NAME, TG_ARN과 목표 임계치만 바꾸면 다른 인프라에도 동일한

검증 로직을 적용할 수 있도록 했습니다.

테스트용 endpoint도 공통으로 정의했습니다. /는 처리 인스턴스 식별, /health는 ALB 헬스체크, /kill은 장애 주입, /work는 CPU 부하 유발에 사용했습니다.

개발 단계에서는 팀 인프라와 별개로 최소 AWS 환경(fixtures/)을 Terraform으로 구성해 장애·복구·스케일링 시나리오를 반복 검증했습니다. 팀 인프라 완성 후 대상 정보만 교체해 동일한 검증 코드를 적용했습니다.

04-2 인프라 상태와 클라이언트 요청을 분리해 검증

검증 기준을 두 관점으로 나눴습니다. pytest와 boto3로 Target Group의 Healthy Target 수, AZ 분산, 인스턴스 교체, 스케일 동작을 확인해 인프라가 의도한 구성과 동작을 보이는지 검증했습니다. Locust로는 외부 요청을 지속적으로 발생시켜 클라이언트 관점의 가용성과 응답 성능을 측정했습니다.

Locust 시나리오는 역할이 다른 세 종류의 가상 사용자로 구성했습니다.

- **ObserveUser** — / 요청으로 응답 시간, 성공 여부, 처리 인스턴스 분포를 기록
- **ScalingUser** — /work 호출로 CPU 부하를 발생시켜 Scale-out 유도
- **FaultUser** — 전체 중 1명만 투입해 /kill로 장애 주입

ObserveUser와 Scaling/FaultUser를 분리해 관측 요청과 부하·장애 주입 요청을 따로 기록했습니다.

04-3 계단식 부하 테스트 기준

계단식 부하에서는 P95 1,000ms 초과를 종료 조건으로 두고, 임계치를 넘은 시점의 사용자 수를 기록했습니다.

04-4 장애·부하 테스트 결과

시나리오	결과
정상 상태	29,093 req · 실패 0 · P95 13ms · 두 인스턴스 50:50 분산
단일 앱 장애	약 40초간 사용자 체감 장애 · 신규 인스턴스 참여까지 3분 46초
전체 앱 장애	완전 서비스 중단 약 232초
Scale-out / in	관측 요청 112,445건 · 실패 0 · P95 16ms · 요청 중단 없음
계단식 부하	20단계(400명)에서 P95 1,113ms · 목표 1,000ms 초과

04-5 검증에서 확인한 차이와 원인

ALB 헬스체크 반영 시차. 애플리케이션 장애 직후에는 ALB Health Check가 반영되기 전까지 장애가 발생한 Target으로 요청이 계속 전달되어 일부 500 오류가 발생했습니다. ALB가 해당 Target을 제외한 뒤에는 실패 응답이 사라졌습니다. 신규 인스턴스의 생성 시점과 실제 요청 처리 참여 시점도 달라 두 시점을 구분해 측정했습니다.

서버 지표와 클라이언트 체감의 차이. Grafana의 서버 측 응답 시간은 거의 변하지 않았으나, 외부에서 측정한 P95는 크게 증가했습니다. 서버 내부 지표만으로는 외부 P95 증가를 설명할 수 없었습니다. 이후 DR 프로젝트에서도 내부 상태와 외부 요청 결과를 분리해 검증했습니다.

Scale-in 미동작 원인. Scaling 테스트에서 Scale-in이 예상대로 발생하지 않았습니다. CloudWatch의 missing data 처리 방식이 원인이었고, 알람 설정 수정이 필요한 부분을 담당자에게 전달했습니다.

프로젝트 간 연결

K3s·Argo CD GitOps 운영 구성 → 멀티클라우드 DR 운영·검증 자동화

- K3s·GitOps 프로젝트에서 사용한 base·overlay, External Secrets, HPA를 DR 프로젝트에도 적용 하되 EKS·AKS와 DR 상태에 맞게 구성을 나눴습니다.
- GitHub App은 K3s·GitOps 프로젝트의 Private Repository 인증에서 도입했고, DR 프로젝트에서는 GitOps 저장소 변경·DR 상태 기록, 애플리케이션 저장소 조회, DR Workflow 실행 요청으로 용도를 나눠 각 작업에 필요한 최소 권한만 부여했습니다.

K3s·Argo CD GitOps 운영 구성 → PR Preview 환경 생성·정리 자동화

- K3s 워크로드 배포·자동화 경험을 PR Preview에 적용해, PR 이벤트에 따라 격리된 Kubernetes 환경을 생성·정리하는 CI/CD를 구현했습니다.

AWS 인프라 검증·장애·부하 테스트 → 멀티클라우드 DR 운영·검증 자동화

- AWS 인프라 검증에서 리소스 상태와 클라이언트 요청 결과의 차이를 확인했고, DR 프로젝트에서는 Traffic Driver로 성공률·응답 시간을 측정하고 Accepted 주문의 최종 DB 반영 여부까지 검증했습니다.

읽어주셔서 감사합니다

김지호

z33hyo@gmail.com · github.com/z-kim